Bài 1.10 – Toán: xác suất cơ bản

> **Giai đoạn 1 – Python cho ML & Data Science** · Thời lượng: 1 buổi (2–3 giờ) · Hình thức mở đầu: *Đố vui trước khi học*

🎯 Sau bài này bạn sẽ

- Hiểu xác suất là gì và **ước lượng xác suất bằng mô phỏng** trong Python.
- Phân biệt P(A | B) với P(B | A), cái bẫy mà cả bác sĩ cũng hay mắc.
- Dùng **định lý Bayes** để tính "email có chữ 'miễn phí' thì khả năng là spam bao nhiêu?"
- Tính **kỳ vọng** để ra quyết định.
- Biết 3 phân phối hay gặp: Bernoulli, Binomial, Uniform (cùng với phân phối chuẩn ở bài 1.7).

> Vì sao người học ML cần xác suất? Vì mô hình phân loại **không trả lời "có/không"**, nó trả lời **"có, với xác suất 87%"**. Hiểu xác suất để đọc đúng và dùng đúng con số đó.

---

1. Đố vui trước khi học 🤔

Một bệnh hiếm gặp ở **1%** dân số. Có một xét nghiệm khá tốt:

- Nếu **có bệnh**, xét nghiệm báo dương tính **99%** trường hợp.
- Nếu **không bệnh**, xét nghiệm vẫn báo nhầm dương tính **5%** trường hợp.

Bạn đi xét nghiệm và nhận kết quả **dương tính**. Khả năng bạn thật sự có bệnh là bao nhiêu?

- A. Khoảng 99%
- B. Khoảng 95%
- C. Khoảng 50%
- D. Khoảng 17%

Ghi lại đáp án. Câu trả lời nằm ở mục 4. (Gợi ý: phần lớn mọi người chọn A hoặc B, và đều sai.)

---

2. Xác suất là gì?

2.1. Khái niệm

- **Probability (Xác suất):** Con số từ 0 đến 1 đo **khả năng một sự việc xảy ra**. 0 = không bao giờ, 1 = chắc chắn.
- **Ví dụ đời thực:** Dự báo "70% khả năng mưa" nghĩa là: trong nhiều ngày có điều kiện thời tiết giống hôm nay, khoảng 70% số ngày đó có mưa.

Ký hiệu: **P(A)** = xác suất sự việc A xảy ra.

Công thức đơn giản nhất (khi mọi khả năng đều như nhau):

```
                số trường hợp A xảy ra
P(A) = ──────────────────────────────
```

```
            tổng số trường hợp
```

Tính thử: Tung xúc xắc, P(ra mặt 6) = 1/6 ≈ 0,1667.

2.2. Xác suất = tỉ lệ khi lặp lại rất nhiều lần

Thay vì tính bằng công thức, ta có thể **mô phỏng**: tung xúc xắc thật nhiều lần và đếm.

```
import numpy as np
import pandas as pd

rng = np.random.default_rng(42)       # seed để kết quả lặp lại được (bài 1.3)
for n in [10, 100, 10_000, 1_000_000]:
    tung = rng.integers(1, 7, size=n)              # tung n lần, mỗi lần ra 1..6
    print(f"{n:>9} lần tung: tỉ lệ ra mặt 6 = {(tung == 6).mean():.4f}")
```

Kết quả mẫu:

```
       10 lần tung: tỉ lệ ra mặt 6 = 0.1000
      100 lần tung: tỉ lệ ra mặt 6 = 0.1300
    10000 lần tung: tỉ lệ ra mặt 6 = 0.1628
  1000000 lần tung: tỉ lệ ra mặt 6 = 0.1663
```

Càng tung nhiều, tỉ lệ càng gần 1/6 = 0,1667. Đây là **Luật số lớn (Law of Large Numbers)**.

Liên hệ ML: Đây là lý do **càng nhiều dữ liệu, mô hình càng đáng tin**. Với 10 mẫu, bạn có thể kết luận sai hoàn toàn (0,10 thay vì 0,167).

2.3. Ba quy tắc cơ bản

Quy tắc	Công thức	Ví dụ
Phần bù	P(không A) = 1 − P(A)	P(không ra 6) = 1 − 1/6 = 5/6
Hoặc (A, B không thể cùng xảy ra)	P(A hoặc B) = P(A) + P(B)	P(ra 5 hoặc 6) = 1/6 + 1/6 = 1/3
Và (A, B **độc lập**)	P(A và B) = P(A) × P(B)	P(tung 2 lần đều ra 6) = 1/6 × 1/6 = 1/36 ≈ 0,0278

- **Independent (Độc lập):** Hai sự việc mà việc này xảy ra **không ảnh hưởng** khả năng xảy ra của việc kia. Ví dụ: lần tung thứ nhất không ảnh hưởng lần tung thứ hai.

---

3. Xác suất có điều kiện

3.1. Khái niệm

- **Conditional probability (Xác suất có điều kiện):** P(A | B), đọc là "xác suất A **khi biết** B đã xảy ra".
- **Ví dụ đời thực:** P(mưa) trong năm ở TP.HCM có thể là 40%. Nhưng P(mưa | trời đang kéo mây đen) cao hơn nhiều. Thông tin mới làm thay đổi xác suất.

Công thức:

$$P(A \mid B) = \frac{P(A \text{ và } B)}{P(B)}$$

Trực giác: Khi biết B đã xảy ra, ta **thu hẹp thế giới** chỉ còn những trường hợp có B, rồi xem trong đó bao nhiêu có A.

3.2. Ví dụ với bảng đếm

Bạn có 100 email đã gán nhãn. Đếm xem email nào có chứa chữ "miễn phí", email nào là spam:

```
emails = pd.DataFrame({
    "co_mien_phi": [1]*18 + [0]*12 + [1]*4 + [0]*66,   # 1 = có chữ "miễn phí"
    "spam":        [1]*18 + [1]*12 + [0]*4 + [0]*66,   # 1 = spam
})
# Bảng chéo (bài 1.4), margins=True thêm dòng/cột tổng "All"
print(pd.crosstab(emails["co_mien_phi"], emails["spam"], margins=True))
```

Kết quả mẫu:

```
spam          0   1  All
co_mien_phi
0            66  12   78
1             4  18   22
All          70  30  100
```

Đọc bảng:

- **P(spam)** = 30/100 = **0,30**.
- **P("miễn phí" | spam)**: chỉ xét 30 email spam (cột 1), có 18 email chứa "miễn phí" → 18/30 = **0,60**.
- **P(spam | "miễn phí")**: chỉ xét 22 email có "miễn phí" (dòng 1), có 18 là spam → 18/22 ≈ **0,818**.

```
p_spam = emails["spam"].mean()                                    # trung
bình cột 0/1 = tỉ lệ
p_mf_neu_spam = emails.loc[emails["spam"] == 1, "co_mien_phi"].mean()     # thu
hẹp về spam
p_spam_neu_mf = emails.loc[emails["co_mien_phi"] == 1, "spam"].mean()     # thu
hẹp về "miễn phí"
print(p_spam, p_mf_neu_spam, round(p_spam_neu_mf, 3))
```

Kết quả mẫu:

```
0.3 0.6 0.818
```

3.3. Cái bẫy: P(A | B) ≠ P(B | A)

- P("miễn phí" | spam) = 0,60: "60% email spam có chữ miễn phí".
- P(spam | "miễn phí") = 0,818: "82% email có chữ miễn phí là spam".

Hai câu nghe na ná, nhưng **khác hẳn nhau**. Một ví dụ dễ thấy hơn: P(có 4 chân | là con chó) ≈ 1, nhưng P(là con chó | có 4 chân) thấp hơn nhiều (vì còn mèo, bò, cái bàn...).

Khi lọc spam, ta **biết** email có chữ gì, và **muốn tính** P(spam | chữ đó). Nhưng dữ liệu dễ thống kê hơn lại là P(chữ đó | spam). Làm sao "lật ngược"? → **Định lý Bayes**.

---

4. Định lý Bayes

4.1. Công thức

$$P(A \mid B) = \frac{P(B \mid A) \times P(A)}{P(B)}$$

Ký hiệu	Tên gọi	Ý nghĩa trong ví dụ spam
P(A)	**Prior (Tiên nghiệm)**	Niềm tin ban đầu: 30% email là spam
P(B | A)	**Likelihood (Khả năng)**	Spam thì 60% có chữ "miễn phí"
P(B)	Bằng chứng	Tỉ lệ email có chữ "miễn phí" nói chung
P(A | B)	**Posterior (Hậu nghiệm)**	Niềm tin **sau khi** thấy chữ "miễn phí"

Tính P(B) bằng cách cộng hai đường dẫn tới B:

$$P(B) = P(B \mid A) \times P(A) \;+\; P(B \mid \text{không } A) \times P(\text{không } A)$$

Trực giác: Bayes là công thức **cập nhật niềm tin khi có bằng chứng mới**. Ban đầu tin 30% là spam. Thấy chữ "miễn phí" → nâng lên 82%.

4.2. Tính thử với spam

Từ bảng: P(spam) = 0,3; P(mf | spam) = 0,6; P(mf | thường) = 4/70 ≈ 0,0571.

- P(mf) = 0,6 × 0,3 + 0,0571 × 0,7 = 0,18 + 0,04 = **0,22**
- P(spam | mf) = 0,18 / 0,22 ≈ **0,818** ✓ Khớp với đếm trực tiếp ở mục 3.2.

4.3. Giải câu đố đầu bài

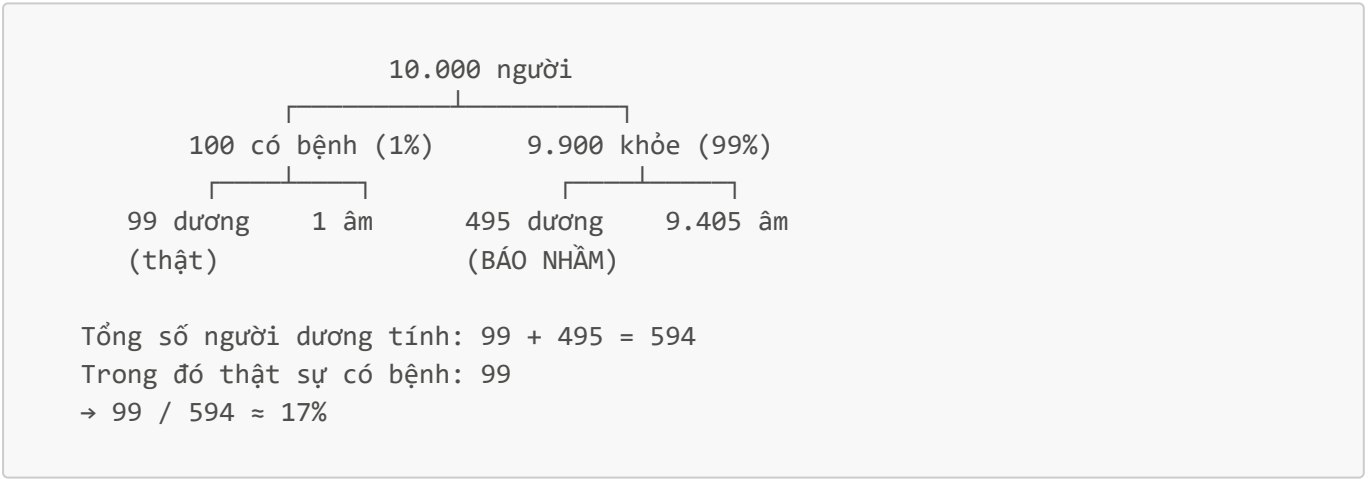
A = có bệnh, B = dương tính.

- P(bệnh) = 0,01 (prior)
- P(dương | bệnh) = 0,99
- P(dương | khỏe) = 0,05

Bước 1: P(dương) = 0,99 × 0,01 + 0,05 × 0,99 = 0,0099 + 0,0495 = **0,0594**

Bước 2: P(bệnh | dương) = 0,0099 / 0,0594 ≈ **0,1667 ≈ 17%**. Đáp án **D**!

Vì sao thấp vậy? Hình dung 10.000 người đi xét nghiệm:



Bệnh quá hiếm, nên dù xét nghiệm chỉ báo nhầm 5%, **số người khỏe bị báo nhầm (495) vẫn nhiều gấp 5 lần số người bệnh thật (99)**.

Kiểm chứng bằng mô phỏng 1 triệu người:

```
n = 1_000_000
benh = rng.random(n) < 0.01                          # rng.random(): số ngẫu nhiên
trong [0, 1); < 0.01 → True với xác suất 1%
duong = np.where(benh,
                 rng.random(n) < 0.99,               # người bệnh: dương tính 99%
                 rng.random(n) < 0.05)               # người khỏe: dương tính nhầm 5%
print(benh.sum(), duong.sum(), (benh & duong).sum(), round(benh[duong].mean(), 4))
```

Kết quả mẫu (số ngẫu nhiên; nếu chạy các ô theo thứ tự khác, kết quả sẽ hơi khác):

```
9847 59615 9737 0.1633
```

Trong 59.615 người dương tính, chỉ 16,3% thật sự có bệnh. Khớp với Bayes (16,7%).

4.4. Liên hệ ML

Bài toán	Hệ quả
Lọc spam	**Naive Bayes** dùng đúng công thức này, nhân lên cho từng chữ trong email (sẽ học kỹ ở bài 2.8)
Phát hiện gian lận thẻ tín dụng (gian lận rất hiếm, như bệnh hiếm)	Mô hình báo "gian lận" thì phần lớn vẫn có thể là báo nhầm. Đây là vấn đề **dữ liệu mất cân bằng**, sẽ gặp lại khi học Precision ở bài 2.5

5. Biến ngẫu nhiên và kỳ vọng

5.1. Khái niệm

- **Random variable (Biến ngẫu nhiên):** Một con số có giá trị phụ thuộc vào may rủi. Ví dụ: kết quả tung xúc xắc, số khách vào quán hôm nay.
- **Expected value (Kỳ vọng):** Giá trị **trung bình về lâu dài** nếu lặp lại rất nhiều lần. Ký hiệu E[X].

Công thức:

$$E[X] = x_1 \cdot p_1 + x_2 \cdot p_2 + \ldots + x_n \cdot p_n$$

Ký hiệu	Ý nghĩa
x_i	Mỗi giá trị có thể xảy ra
p_i	Xác suất của giá trị đó (tổng các $p_i = 1$)

Để ý: đây chính là **tích vô hướng** (bài 1.8) giữa vector giá trị và vector xác suất!

Tính thử với xúc xắc: E = 1×1/6 + 2×1/6 + ... + 6×1/6 = 21/6 = **3,5**. Bạn không bao giờ tung ra 3,5, nhưng trung bình nhiều lần tung sẽ là 3,5.

5.2. Ví dụ ra quyết định: có nên bán "hộp quà bí ẩn"?

Shop của bạn bán hộp quà bí ẩn. Trong hộp có thể là:

Phần thưởng	Giá trị (đ)	Xác suất
Không có gì	0	90%
Voucher	50.000	9%
Tai nghe	2.000.000	1%

Bán mỗi hộp 30.000đ thì shop lời hay lỗ về lâu dài?

```
gia_tri = np.array([0, 50_000, 2_000_000])
xac_suat = np.array([0.90, 0.09, 0.01])
print(gia_tri @ xac_suat, xac_suat.sum())    # kỳ vọng = tích vô hướng; kiểm tra tổng xác suất = 1
```

Kết quả mẫu:

```
24500.0 1.0
```

Mỗi hộp shop trung bình phải "trả" 24.500đ giá trị quà. Bán 30.000đ → lời trung bình **5.500đ/hộp**.

Liên hệ ML: Hàm loss bạn tối thiểu hóa (bài 1.9) thật ra là **kỳ vọng của sai số** trên dữ liệu. Và nhiều quyết định kinh doanh dựa trên ML (cho vay hay không, giảm giá cho khách nào) đều quy về: **"chọn phương án có kỳ vọng lợi nhuận cao nhất"**.

---

6. Ba phân phối hay gặp

Phân phối	Mô tả	Ví dụ	NumPy
Bernoulli	Một lần thử, chỉ có 2 kết quả: 1 (thành công, xác suất p) hoặc 0	Khách có click quảng cáo không? Email có phải spam không?	`rng.binomial(n=1, p=...)`
Binomial (Nhị thức)	Đếm số lần thành công trong n lần thử Bernoulli độc lập	100 khách vào shop, mỗi người mua với xác suất 3%. Hôm nay bao nhiêu người mua?	`rng.binomial(n=100, p=0.03)`
Uniform (Đều)	Mọi giá trị trong khoảng [a, b] có khả năng như nhau	Khởi tạo trọng số ngẫu nhiên cho mô hình	`rng.uniform(a, b)`
Normal (Chuẩn)	Hình chuông (đã học ở bài 1.7)	Chiều cao, sai số đo	`rng.normal(...)`

```
print(rng.binomial(n=1, p=0.5, size=10))       # 10 lần tung đồng xu: 1 = ngửa
so_khach_mua = rng.binomial(n=100, p=0.03, size=7)   # 7 ngày, mỗi ngày 100 khách, xác suất mua 3%
print(so_khach_mua, so_khach_mua.mean())
print(rng.uniform(0, 1, size=5).round(3))      # 5 số đều trong [0, 1)
```

Kết quả mẫu (số ngẫu nhiên):

```
[1 0 1 1 1 0 1 1 0 1]
[2 5 3 2 5 5 3] 3.5714285714285716
[0.424 0.482 0.741 0.415 0.718]
```

Kỳ vọng của Binomial là n × p = 100 × 0,03 = **3** khách/ngày. Trung bình 7 ngày mô phỏng là 3,57: gần 3, nhưng 7 ngày là ít nên còn lệch (Luật số lớn!).

Liên hệ ML: Cột đích của bài toán phân loại 2 lớp (spam/không, trả nợ/không) chính là biến **Bernoulli**. Logistic Regression (bài 2.4) dự đoán tham số p của nó.

---

7. Bài tập

1. **Bayes – gian lận thẻ.** 2% giao dịch là gian lận. Hệ thống phát hiện đúng 95% giao dịch gian lận, nhưng báo nhầm 10% giao dịch bình thường. Một giao dịch bị báo "gian lận". Xác suất nó thật sự gian lận là bao nhiêu? Vẽ cây 10.000 giao dịch như mục 4.3.
2. Tung xúc xắc 4 lần. Xác suất **có ít nhất một lần** ra mặt 6? (Gợi ý: dùng phần bù: 1 − P(không lần nào ra 6).) Kiểm tra bằng mô phỏng 1 triệu lần.
3. **Kỳ vọng – cho vay.** Ngân hàng cho vay 1 khoản. Nếu khách trả nợ (xác suất 80%), ngân hàng lời 500.000đ. Nếu khách vỡ nợ (20%), ngân hàng lỗ 3.000.000đ. Có nên cho vay không? Xác suất trả nợ tối thiểu phải là bao nhiêu thì mới hòa vốn?
4. Bảng email ở mục 3.2: tính P("miễn phí" | thường) và P(thường | "miễn phí").

Đáp án

> Mẹo Notion: chọn tiêu đề "Đáp án" → `Turn into → Toggle heading 3`.

1. P(báo) = 0,95 × 0,02 + 0,10 × 0,98 = 0,019 + 0,098 = 0,117. P(gian lận | báo) = 0,019 / 0,117 ≈ **0,1624 ≈ 16%**.

```
                10.000 giao dịch
            ┌──────────┴──────────┐
      200 gian lận            9.800 bình thường
        ┌─────┴─────┐           ┌─────┴─────┐
   190 báo   10 lọt        980 báo nhầm   8.820 OK
→ Bị báo: 190 + 980 = 1.170, trong đó gian lận thật: 190 → 16%
```

Nghĩa là nhân viên kiểm tra sẽ phải xem 6 giao dịch bị báo mới gặp 1 vụ gian lận thật.

2. $1 - (5/6)^4 = 1 - 0{,}4823 \approx$ **0,5177**.

```
tung = rng.integers(1, 7, size=(1_000_000, 4))     # 1 triệu lượt, mỗi lượt tung 4
lần
print(((tung == 6).any(axis=1)).mean())            # ≈ 0.517; any(axis=1): có ô
nào True trong hàng không
```

3. Kỳ vọng = 0,8 × 500.000 + 0,2 × (−3.000.000) = 400.000 − 600.000 = **−200.000đ** → **lỗ**, không nên cho vay. Hòa vốn khi p × 500.000 = (1 − p) × 3.000.000 → 3.500.000p = 3.000.000 → **p ≈ 0,857**. Khách phải có xác suất trả nợ trên 85,7%. Mô hình chấm điểm tín dụng (Project 1) chính là để ước lượng con số p này cho từng khách.

4. P(mf | thường) = 4/70 ≈ **0,057**. P(thường | mf) = 4/22 ≈ **0,182** (= 1 − 0,818).

---

8. Flashcard

Mặt trước	**Mặt sau**
Xác suất nằm trong khoảng nào?	Từ 0 (không bao giờ) đến 1 (chắc chắn).
Luật số lớn?	Lặp càng nhiều, tỉ lệ thực tế càng gần xác suất thật.
P(A và B) khi A, B độc lập?	P(A) × P(B)
P(A | B) nghĩa là?	Xác suất A khi đã biết B xảy ra.
P(A | B) có bằng P(B | A)?	Không! (Chó có 4 chân ≠ có 4 chân là chó.)
Định lý Bayes?	P(A | B) = P(B | A) × P(A) / P(B)
Prior vs Posterior?	Prior: niềm tin trước bằng chứng. Posterior: sau bằng chứng.
Vì sao xét nghiệm tốt vẫn báo nhầm nhiều với bệnh hiếm?	Người khỏe quá đông, 5% của số đông vẫn lớn hơn số người bệnh.
Kỳ vọng tính thế nào?	Tổng (giá trị × xác suất) = tích vô hướng.
Cột đích phân loại 2 lớp là phân phối gì?	Bernoulli.

Bài tiếp theo: 1.11 – Chuẩn hóa dữ liệu, chia train/validation/test.